

Entropy

Entropy is a Measure of the amount of uncertainty, randomness, or disorder in a dataset.

It is widely used in Decision Trees to determine how mixed the classes are within a node.

Entropy tells us.

"How Uncertain are we about the class of a randomly selected sample?"

- More Uncertainty -> Higher Entropy
- Less Uncertainty -> Lower Entropy

Example: Student Pass/Fail Prediction

Case 1: Mixed Classes

Pass	Fail
100%	0%

Every Student belongs to the same class.

- No Uncertainty
- Entropy is zero.

Properties

- Entropy=0 -> Pure Node
- Entropy=1 -> Maximum disorder (binary class)
- Lower Entropy -> Better node

Information Gain(IG) measures how much information is gained after splitting a dataset.

In Simple Terms:

****Information Gain tells us how much uncertainty**

Why is Information Gain Important?

The Goal of a Decision tree is to create pure nodes.

Information Gain Helps the tree decide:

- Which feature should be used for splitting?
- which split reduces uncertainty the most?

The Split with the highest Information gain is selected.

Pruning in Decision trees

Pruning is the process of removing unnecessary branches from a Decision Tree to make it simpler and prevent overfitting.

A Decision Tree can continue splitting until it perfectly fits the training data. while this may produce very high training accuracy, it often

Memorization

- Remembers answers without understanding concepts.
- Performs well on familiar questions.
- Struggles with new Questions.

This is Similar to Overfitting.

Understanding

- Learns concepts instead of memorizing
- Can Solve new Problems.

Without Pruning?

- Very Deep Tree
- High Training Accuracy.
- Poor test Accuracy.
- Overfitting.

With Pruning

- Simpler Tree
- Slightly Lower Training Accuracy.
- Better test accuracy.
- Better Generalization.

Overfitted Tree

Types of Pruning

1. Pre-Pruning(Early Stopping) In Pre-Pruning

Common Parameters

Parameters	Purpose
max_depth	Maximum depth of Tree
min_samples_split	minimum samples needed to split a node
min_samples_leaf	Minimum Samples required in a leaf node

Parameters	Purpose
max_leaf_nodes	Maximum Number of Leaf Nodes

Example

```
from sklearn.tree import DecisionTreeClassifier

model=DecisionTreeClassifier(
    max_depth=3,
    min_samples_leaf=5
)
```

2. Post-Pruning

In Post-Pruning, we first grow the complete tree.

Then we remove branches that do not improve model Performance.

Process:

1. Build the full tree
2. Evaluate Branches
3. Remove branches with little contribution.
4. Keep out Useful Splits.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

# Load Dataset
iris=load_iris()
X=iris.data
y=iris.target

# Create Decision Tree Model
dt=DecisionTreeClassifier(criterion="entropy",max_depth=3)

# Train Model
dt.fit(X,y)

# Prediction
print(dt.predict([[13.5,13.0,11.1,10.2]]))

[2]
```

2. Train Multiple Models

We Train a decision Tree for each sample.

3. Make Predictions

4. Combine Results

- Classification -> Majority Voting
- Regression -> Average

Why Bagging?

- Reduces Overfitting
- Makes model More stable.
- Improves Accuracy.
- Mistakes are cancel out

Feature Importance (Random Forest)

Feature Importance Tells us:

- Which Features are most useful for prediction.
- Some Features help Prediction a lot
- Some Features don't help much.

Random Forest Finds this Automatically.

How it Decides?

- If a feature helps to split data better (less impurity) ,it is important.

Output

- Value between 0 and 1
- Higher Value = more important feature

Out of Bag Error

- OOB error is a way to check model accuracy without using a separate test set.

In Random Forest:

- Each tree is trained using a random sample of data.
- Some data points are not used for that tree.

These unused data points are called Out-of-Bag(OOB) Samples.

How it Works

1. Train many trees using random samples.
2. Each tree leaves out some data.
3. Use the left-out data to test that tree
4. Combine all errors from all trees.

Final Result

- Average of all these errors=OOB Error

```
from sklearn.datasets import load_iris

from sklearn.ensemble import RandomForestClassifier

import pandas as pd

import numpy as np

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

# Create Random Forest model

rf = RandomForestClassifier(

    n_estimators=100,    # number of trees

    oob_score=True,     # enable Out-of-Bag error

    random_state=42

)

# Train model

rf.fit(X, y)

# 1. OOB Score (Out-of-Bag Accuracy)

print("OOB Accuracy:", rf.oob_score_)

print("OOB Error:", 1 - rf.oob_score_)

# 2. Feature Importance

importance = pd.DataFrame({

    "Feature": iris.feature_names,

    "Importance": rf.feature_importances_

})
```

```
importance = importance.sort_values(by="Importance", ascending=False)
```

```
print("\nFeature Importance:")
```

```
print(importance)
```

```
# 3. Individual Prediction Example
```

```
sample = np.array([[5.1, 3.5, 1.4, 0.2]])
```

```
print("\nPrediction:", rf.predict(sample))
```

```
OOB Accuracy: 0.9533333333333334
```

```
OOB Error: 0.046666666666666634
```

```
Feature Importance:
```

	Feature	Importance
2	petal length (cm)	0.436130
3	petal width (cm)	0.436065
0	sepal length (cm)	0.106128
1	sepal width (cm)	0.021678

```
Prediction: [0]
```

Hyperplane

A Hyperplane is a Decision Boundary that separates different classes.

Example

Student_Data

Hyperplane in Different Dimensions

Number of Features	Hyperplane
2 Features	

2. Accuracy

Accuracy measures the proportion of correct predictions.

Formula

3. Precision

Precision Measures: Out of all Predicted Positives, how many are actually positive?

Formula

Precision = $TP / (TP + FP)$

Example:

Precision= $40/(40+5) = 88.89\%$

4. Recall

Recall Measures:

Out of all actual positives, how many were correctly identified?

Formula:

Recall= $TP/(TP+FN)$

Example:

Recall= $40/(40+10) = 80\%$

5.F1 Score

F1 Score balances Precision and Recall.

Formula

$F1 = 2 \times$

6.ROC Curve

Receiver Operating Characteristic Curve

AUC	Meaning
0.0	Worst Model
0.5	Random Guessing
0.7	Fair
0.8	Good
0.9	Excellent
1.0	Perfect Model

1. IMPORT LIBRARIES

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_auc_score
)
```

```

import numpy as np

# 2. LOAD DATASET
iris = load_iris()
X = iris.data
y = iris.target

# 3. TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# 4. CREATE SVM MODEL

# kernel options: 'linear', 'rbf', 'poly', 'sigmoid'
svm = SVC(
    kernel='rbf',      # non-linear kernel
    probability=True,  # needed for ROC-AUC
    random_state=42
)

# 5. TRAIN MODEL
svm.fit(X_train, y_train)

# 6. PREDICTION
y_pred = svm.predict(X_test)

# probability scores (for ROC-AUC)
y_prob = svm.predict_proba(X_test)

# 7. EVALUATION METRICS
print("MODEL PERFORMANCE")

# Accuracy
acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)

# Precision
precision = precision_score(y_test, y_pred, average='macro')
print("Precision:", precision)

# Recall
recall = recall_score(y_test, y_pred, average='macro')
print("Recall:", recall)

# F1 Score
f1 = f1_score(y_test, y_pred, average='macro')
print("F1 Score:", f1)

# Confusion Matrix

```



```

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# ROC-AUC (multi-class)
roc = roc_auc_score(y_test, y_prob, multi_class='ovr') #One-vs-Rest approach
print("\nROC-AUC Score:", roc)

# 8. SINGLE PREDICTION
sample = np.array([[5.1, 3.5, 1.4, 0.2]])
prediction = svm.predict(sample)
print("\nSingle Sample Prediction:", prediction)

MODEL PERFORMANCE
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

ROC-AUC Score: 1.0

Single Sample Prediction: [0]

```

K-Means Clustering

K-Means is an unsupervised learning algorithm used to group similar data points into clusters.

Goal

Divide data into **K Groups (clusters)** where:

- Points in the same cluster are similar
- Points in different Clusters are different.

How K-Means Works (Steps)

1. Choose the number of clusters **K**
2. Randomly initialize **K Centroids**
3. Assign each data point to the nearest centroid.
4. Recalculate the centroid (mean of cluster points).
5. Repeat Steps 3 and 4 until centroids stop changing.

```

import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np

```

```

import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# 1. Load dataset
iris = load_iris()
X = iris.data

# 2. Feature scaling (important for K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

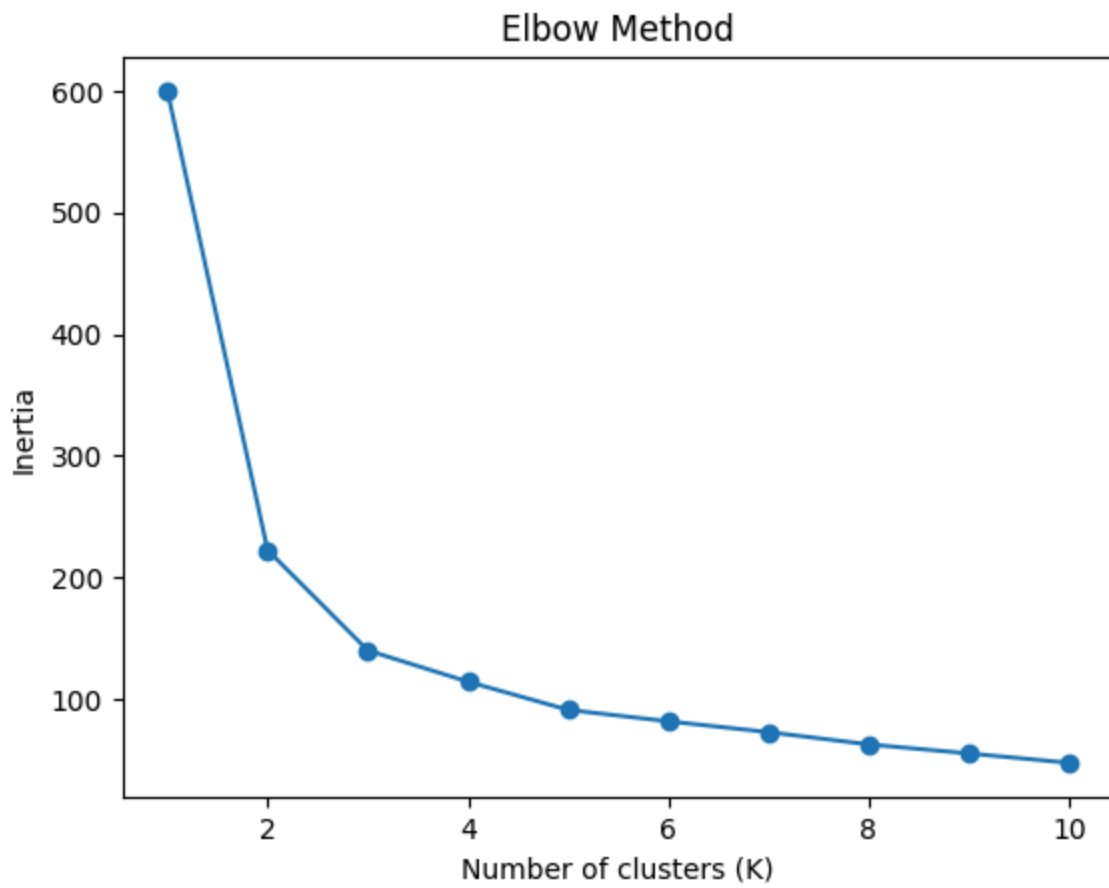
# 3. Elbow Method (Inertia)
inertia = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
plt.plot(K_range, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.show()

# 4. Fit final KMeans (choose K = 3 for iris)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# 5. Silhouette Score
score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)

# 6. Visualize clusters (first 2 features)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            s=300, c='red', marker='X')
plt.title("K-Means Clustering Result")
plt.show()

```



Silhouette Score: 0.45994823920518635



Hierarchical Clustering

Hierarchical Clustering is an Unsupervised learning algorithm that groups similar data points into clusters by building a hierarchy of clusters.

Unlike K-Means, You dont

How Hierarchical Clustering Works

There are two approaches:

1. Agglomerative Clustering (Bottom-up)

- Start with each data point as its own cluster.
- Merge the two closest clusters
- Repeat until all points belong to one cluster.

2. Divisive Clustering (Top-Down)

- Start with all data points in one cluster.
- split clusters recursively
- Continue until each point forms its own cluster.

Dendogram

A Dendogram is a tree-like diagram that shows how clusters are merged.

- Each Leaf represents a data point.
- Branches show cluster merging.
- Height represents distance between clusters.

Example:

Linkage Methods

Linkage Determines how the distance between clusters is calculated.

1. Single Linkage

Uses the minimum distance between the clusters.

Distance = Minimum Distance between any two points

Pros

- Finds irregular-shaped clusters

Cons

- Sensitive to noise

2. Complete Linkage

Uses the maximum distance between clusters.

Distance = Maximum distance between any two points

Pros

- Creates Compact Clusters

Cons

- Sensitive to Outliers.

3. Average Clusters

Uses the average distance between all points.

Distance = Average Pairwise Distance

Pros

- Balanced Clustering

4. Ward Linkage (Most Popular)

Merges clusters that minimize variance.

Pros

- Produces well-balanced clusters
- Often Gives the best results.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
```

```
# Load dataset
iris = load_iris()
```

[illegible]

